초음파 센서 기반 주차장 차단기 구현

Sigmoid와 Smoothstep을 이용한 서보 제어 비교

정보보안암호수학과 20242089 안승민 | 작성일: 2026년 10월 11일

1 구현 목적 및 함수 선정 과정

1.1 구현 목적과 설계 조건

본 구현은 초음파 센서로 차량 역할의 물체를 감지하고, 서보암을 차단기 바처럼 올리고 내리는 시스템이다. 기존 실습의 서보 왕복 제어에 거리 기반 판단을 추가하고, 선형 각도 변화 대신 S자 형태의 보간 함수를 적용하였다. 차량 접근 시 개방하고, 차량이 지나간 것으로 판단되면 폐쇄하며, 폐쇄 중 재접근하면 다시 개방한다.

현재 소스의 설정은 서보 신호 핀 10번, 초음파 TRIG 12번, ECHO 13번이다. 닫힘 각도는 $0°$, 열림 각도는 $120°$ 이며, 각 이동의 설정 시간은 $T = 1.5\,\mathrm{s}$이다. 이 보고서의 결과는 소스 코드와 수식 분석에 근거한다. 실제 서보의 각도, 진동 및 응답 시간을 측정한 실험 자료는 포함하지 않는다.

1.2 시간을 각도로 변환하는 공통 구조

이동 시작 시각을 t_0, 현재 시각을 t라 하면 정규화된 진행률은 다음과 같다. 코드에서는 경과 시간이 이동 시간을 넘으면 $u = 1$로 제한한다.

$$u = \min\left(\frac{t - t_0}{T}, 1\right), \qquad \theta(u) = \theta_0 + (\theta_1 - \theta_0) f(u), \quad 0 \le u \le 1. \tag{1}$$

여기서 θ_0는 이동 시작 각도, θ_1은 목표 각도이다. 열기와 닫기에 같은 식을 사용하고, 각도 차의 부호로 이동 방향을 결정한다. 기존 선형 보간 $f(u) = u$는 이동 중 속도가 일정하지만 정지 구간과 연결할 때 속도가 갑자기 변한다. 따라서 시작과 끝에서는 느리고 중앙에서는 빠른 함수를 비교 대상으로 정했다.

1.3 Smoothstep 함수의 개발 과정

Sigmoid 외의 함수는 시작과 끝의 위치를 정확히 맞추고, 정지 상태와 연결되는 양 끝의 속도가 0이 되도록 설계하였다. 이를 위해 $p(u) = au^3 + bu^2 + cu + d$에 다음 네 조건을 부여하였다.

$$p(0) = 0, \quad p(1) = 1, \quad p'(0) = 0, \quad p'(1) = 0. \tag{2}$$

첫째와 셋째 조건으로 $d = c = 0$이고, 나머지 조건은 $a + b = 1, 3a + 2b = 0$이다. 따라서 $a = -2, b = 3$이므로 다음 식을 얻는다.

$$\boxed{p(u) = 3u^2 - 2u^3 = u^2(3 - 2u)} \tag{3}$$

이 함수는 3차 Smoothstep이며, 위 네 경계 조건을 만족하는 최소 차수의 다항식이다. $p'(u) = 6u(1 - u) \ge 0$이므로 구간 안에서 역방향 이동이나 목표 각도 초과 없이 단조롭게 이동한다. 또한 곱셈과 뺄셈으로 계산할 수 있어 구현이 간단하다. 다만 $p''(0) = 6, p''(1) = -6$이므로 정지 구간과 연결할 때 가속도까지 연속이 되는 함수는 아니다.

2 Sigmoid 함수와 특성 비교

2.1 Sigmoid의 구간 설정과 정규화

기본 Logistic 함수 $\sigma(x) = 1/(1+e^{-x})$는 무한 구간에서 0과 1에 접근한다. 유한한 이동 시간에 적용하기 위해 입력을 $x = 12(u-0.5)$로 두어 $[-6, 6]$ 구간을 사용하였다. 이때 양 끝의 출력은 정확한 0과 1이 아니므로 코드에서는 다음과 같이 정규화한다.

$$\boxed{s(u) = \frac{\sigma(12(u-0.5)) - \sigma(-6)}{\sigma(6) - \sigma(-6)}} \tag{4}$$

이 변환으로 수학적으로 $s(0) = 0$, $s(1) = 1$이 된다. 또한 코드에서는 이동 시간이 끝나면 명령 각도를 목표 각도로 직접 대입한다. 따라서 부동소수점 오차가 남더라도 마지막 서보 명령은 목표 각도에 맞춰진다.

2.2 변위 및 속도 특성

두 함수 모두 중앙에서 $f(0.5) = 0.5$이며 중앙 대칭의 S자 곡선을 만든다. 하지만 현재 설정의 Sigmoid는 변화가 중앙에 더 집중되고, Smoothstep은 더 넓은 시간 구간에 걸쳐 각도가 변한다.

비교 항목	정규화 Sigmoid	3차 Smoothstep
시작 / 끝 출력	0 / 1	0 / 1
시작 / 끝 기울기	약 0.02975	0
중앙 최대 기울기	약 3.01491	1.5
변화 양상	중앙에 이동 집중	상대적으로 완만한 변화
함수 계산	지수 함수 포함	다항식 연산
정지 구간과의 연결	작은 속도 불연속 존재	속도 연속, 가속도 불연속

정규화된 기울기를 실제 시간에 대한 명령 각속도로 환산하면 $\dot{\theta} = (\theta_1 - \theta_0) f'(u)/T$이다. 전체 개방($0^\circ \to 120^\circ$, 1.5초)의 연속 곡선 기준 최대 명령 각속도는 Sigmoid가 약 $241.2^\circ/\mathrm{s}$, Smoothstep이 $120.0^\circ/\mathrm{s}$이다. 이는 서보가 실제로 달성한 속도가 아니라 보간식이 요구하는 속도이다. 서보의 속도 한계나 부하에 따라 실제 이동은 이 곡선과 다를 수 있다.

진행률 u	경과 시간(s)	Sigmoid 각도(°)	Smoothstep 각도(°)
0.00	0.000	0.00	0.00
0.25	0.375	5.42	18.75
0.50	0.750	60.00	60.00
0.75	1.125	114.58	101.25
1.00	1.500	120.00	120.00

표는 정수 반올림 전의 이론값이다. 닫힘 동작에서는 같은 진행률에 대해 각도가 $120[1 - f(u)]$가 되므로 동일한 속도 특성이 반대 방향으로 나타난다.

2.3 선정 결과와 해석

정지 상태에서 부드럽게 출발하고 멈추는 목적에는 끝점 속도가 0이고 최대 요구 속도가 낮은 Smoothstep이 적합하다. 반면 현재 Sigmoid는 이동 초반과 후반을 더 느리게, 중앙을 더 빠르게 만드는 특성이 뚜렷하다. 이 비교는 기울기 계수 12인 현재 설정에 한정되며 모든 Sigmoid에 일반화할 수 없다. Smoothstep 자체의 계산은 단순하지만, 현재 비교용 코드는 두 함수를 매번 모두 계산하므로 모드 변경만으로 전체 연산량이 줄어드는 구조는 아니다.

3 코드 구현 설명 및 결과 해석

3.1 거리 측정과 차량 감지

USS_measure()는 TRIG에 10 μs 펄스를 출력하고, pulseIn()으로 ECHO의 HIGH 유지 시간을 측정한다. 음속을 346 m/s로 두면 거리 변환식은 $d[\text{mm}] = \tau[\mu\text{s}] \times 0.001 \times 346/2$이다. 거리 측정은 60ms 간격을 기준으로 수행하며, ECHO 대기는 최대 12ms로 제한한다.

유효 거리가 200mm 이하이면 차량 감지 상태로, 250mm 이상이거나 무응답이면 미감지 상태로 전환한다. 200mm 초과 250mm 미만에서는 이전 상태를 유지한다. 이처럼 진입과 해제 기준을 다르게 둔 히스테리시스로 경계 부근의 반복 전환을 줄였다. 감지될 때마다 lastPresenceTime을 갱신하고, 마지막 감지 이후 1초가 지났으며 개방 이동까지 끝난 경우에만 폐쇄를 시작한다.

3.2 상태 전환과 서보 갱신

startMove()는 현재 명령 각도를 시작점으로 저장하고 목표 각도와 시작 시각을 설정한다. gateOpen은 이동 목표가 열림인지를, moving은 이동 진행 여부를 나타낸다. 따라서 gateOpen만으로 실제 개방 완료를 판단하지 않는다.

```
startAngle = currentAngle;
stopAngle = open ? OPEN_ANGLE : CLOSED_ANGLE;
movingMode = selectedMode;
moveStartTime = now;
moving = true;
```

폐쇄 도중 차량을 다시 감지하면 이 함수를 개방 방향으로 호출한다. 현재 각도에서 새 곡선을 시작하므로 명령 위치는 이어지지만, 이전 속도까지 이어받지는 않는다. 재개방에도 새로운 1.5초를 부여하므로 남은 이동 각도가 작으면 명령 속도도 낮아진다.

updateServo()는 약 20ms 간격으로 진행률과 두 함수의 각도를 계산하고, 선택한 각도를 정수로 반올림해 Servo.write()에 전달한다.

```
float t = elapsed >= MOVING_TIME ? 1.0f
          : (float)elapsed / MOVING_TIME;
float sigmoidAngle = startAngle + (stopAngle-startAngle)*sigmoid(t);
float smoothstepAngle = startAngle + (stopAngle-startAngle)*smoothstep(t);
currentAngle = movingMode == 1 ? sigmoidAngle : smoothstepAngle;
myServo.write((int)(currentAngle + 0.5f));
```

긴 delay() 없이 millis()의 경과 시간으로 이동을 관리하므로 이동 중에도 거리 판단을 수행할 수 있다. 다만 pulseIn()의 대기와 시리얼 출력 때문에 20ms 갱신 주기가 엄밀히 보장되지는 않는다. 또한 정수 각도 명령이므로 수학적으로 연속인 곡선도 실제 출력에서는 계단 형태로 근사된다.

3.3 비교 방법과 구현 범위

57600 baud의 시리얼 모니터에서 1을 입력하면 Sigmoid, 2를 입력하면 Smoothstep을 선택한다. 선택값은 이동 시작 시 복사되므로 이동 도중 모드를 변경해도 진행 중인 곡선은 바뀌지 않는다. 시리얼 플로터의 sigmoid, smoothstep은 두 예상 각도이고, actual은 선택한 명령 각도이다. 동일한 접근·제거 조건에서 각 모드의 개방과 폐쇄를 비교할 수 있다.

코드상 차량 감지, 지연 폐쇄, 재감지 시 개방 및 두 보간 함수의 선택 구조가 구현되어 있다. 다만 단일 센서의 미감지만으로 통과를 추정하므로 실제 통과와 센서 무응답을 구분하지 못한다. 물리적인 부드러움과 이동 완료 시간은 서보 사양과 부하를 포함한 별도 실험으로 확인해야 한다.